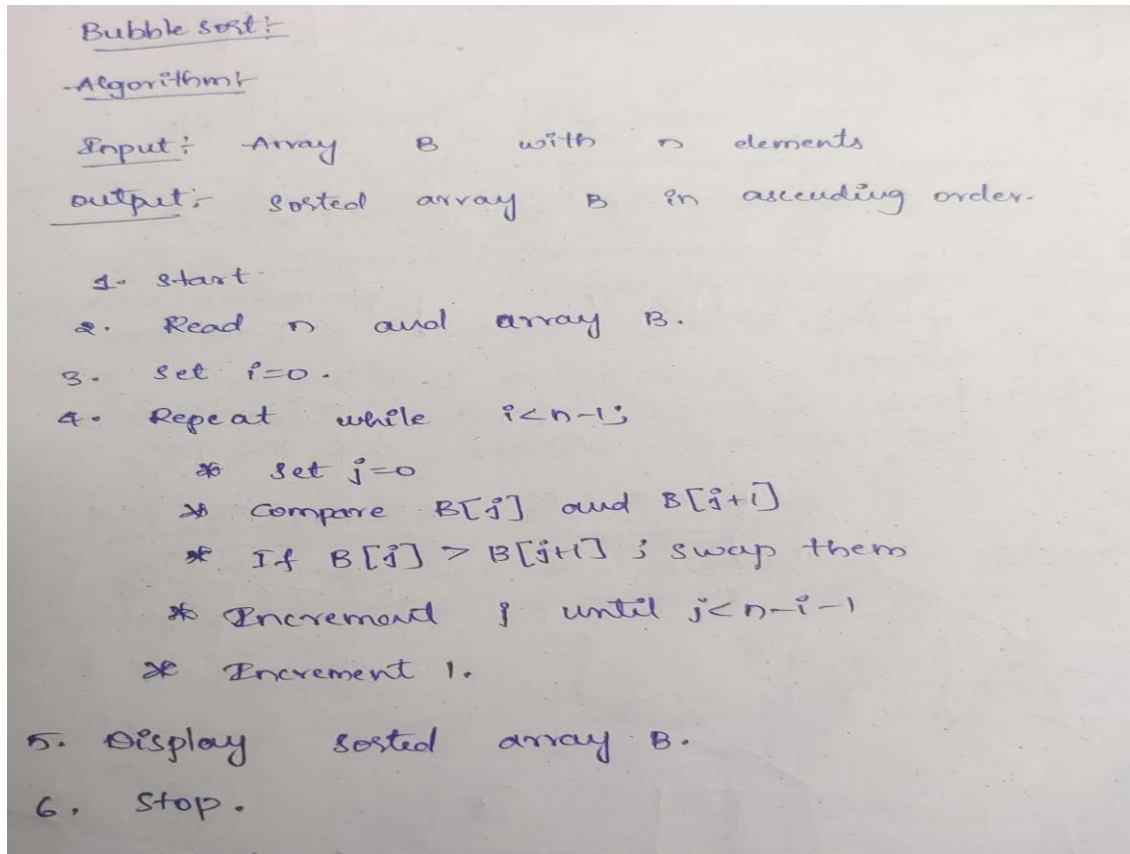


Aim: To Implementation and time analysis of bubble sort algorithm.

Algorithm for bubble sort:



Program of bubble sort:

```
#include <iostream>
using namespace std;

class BubbleSortApp
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;
```

```
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
}

void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

// Bubble Sort Function
void sort()
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

};

int main()
{
    BubbleSortApp bs;
```

```
bs.arrayInput();

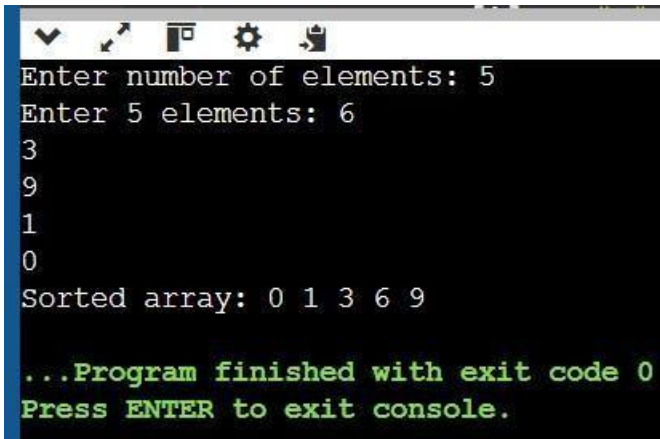
cout << "\nArray before sorting: ";
bs.printArray();

bs.sort();

cout << "\nArray after Bubble Sort: ";
bs.printArray();

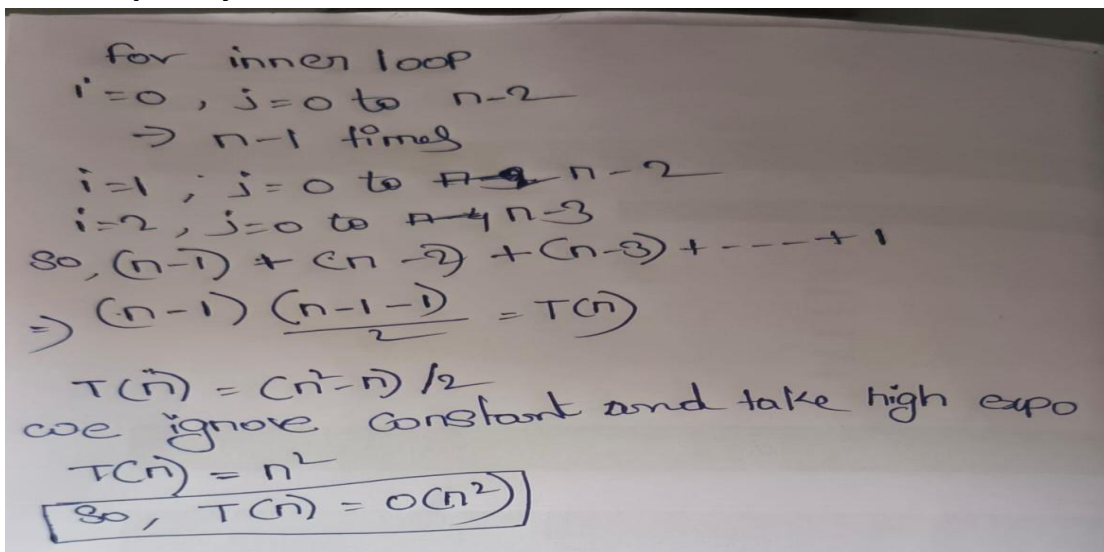
return 0;
}
```

Output:



```
Enter number of elements: 5
Enter 5 elements: 6
3
9
1
0
Sorted array: 0 1 3 6 9
...Program finished with exit code 0
Press ENTER to exit console.
```

Time complexity:



for inner loop
 $i=0, j=0$ to $n-2$
 $\rightarrow n-1$ times
 $i=1, j=0$ to $n-2$
 $i=2, j=0$ to $n-3$
So, $(n-1) + (n-2) + (n-3) + \dots + 1$
 $\Rightarrow \frac{(n-1)(n-1)}{2} = T(n)$
 $T(n) = \frac{(n^2 - n)}{2}$
we ignore constant and take high expo
 $T(n) = n^2$
 $\boxed{\text{So, } T(n) = O(n^2)}$